



Natural language processing course project report

Iztok Bajcar, Urban Vesel and Luka Volk

Abstract

We are developing an AI assistant for the UL FRI student office that answers administrative questions in Slovenian and English based on official faculty documents. At this stage, we implemented an end-to-end pipeline for (i) collecting public UL/FRI web content, (ii) parsing heterogeneous documents (HTML, PDF, DOCX) into structured text, (iii) section-aware chunking, (iv) multilingual embedding and FAISS indexing, and (v) retrieval and optional answer generation via instruction-tuned LLMs. The system supports dense retrieval, optional BM25+dense hybrid retrieval, and optional cross-encoder reranking. For evaluation, we maintain a JSONL question set whose questions are annotated with the relevant chunk identifiers, enabling standard information retrieval metrics (Recall@k, Precision@k, MRR@k, nDCG@k) and basic end-to-end QA proxies such as keyword hit rate, together with retrieval and generation latency measurements.

Keywords

Retrieval augmented generation, dense retrieval, hybrid retrieval, multilingual embeddings, student affairs chatbot

Advisors: Slavko Žitnik

Introduction

The purpose of this project is to develop an AI assistant for the student office of the Faculty of computer science and Informatics of the University of Ljubljana.

It serves as a chatbot that can answer students' questions about the administrative aspects of study programmes, such as study regulations, as well as provide information about the work of the faculty's student office.

Related work that will aid us in the development of the project includes:

- An IFIP conference paper from 2020 by Adamopoulou and Moussiades, providing an overview of chatbot technology that might be useful in determining the appropriate methodology and approaches [1]
- An article by Lewis et al. about RAG [2]
- AN IEEE article from 2017 describing chatbots for university-related FAQs, which aligns with our problem domain [3]
- A paper presenting BARKPLUG V.2, a Large Language Model (LLM)-based chatbot system built using Retrieval Augmented Generation (RAG) pipelines [4]

- A paper that implements and compares multiple RAG pipeline configurations, combining retrieval methods like Dense, MMR, and Hybrid, and chunking strategies like Semantic and Recursive. [5]
- A study proposing the use of local LLMs like Microsoft Phi3, Meta LLaMA3, and Mistral; to power a chatbot for prospective students at Nottingham Trent University's open days. [6]
- A study that compares unsupervised finetuning and RAG across a variety of knowledge intensive tasks [7]
- A survey that synthesizes 51 studies across educational goals, learning contexts, and application scenarios, covering technical components of RAG, identifying key challenges like hallucination, outdated knowledge, and limited multimodality [8]

Methods

Pipeline overview

At this stage, our implementation provides a complete data-to-index-to-answer pipeline, together with scripts for retrieval testing and evaluation. The workflow is run-based: each run stores its raw downloads, processed JSONL files (parsed documents and chunks), and the final FAISS index.

Data collection

We collect publicly available content from UL FRI and other UL domains using a lightweight crawler that starts from a curated list of seed URLs. The crawler performs breadth-first traversal with domain-specific depth limits (separately for `fri.uni-lj.si`, other `*.uni-lj.si` domains, and external domains). It downloads HTML pages as well as linked PDF/DOCX attachments; discovered attachments do not increase the crawl depth. Each downloaded file is stored under the run's `raw/` directory using a stable URL-derived path, and a `manifest.jsonl` file stores provenance (source URL, download time, content type, and SHA-256 hash).

Document parsing and normalization

Downloaded documents are parsed into a common JSON structure consisting of a document identifier (`doc_id`), title, language, and a list of named sections. We support:

- **HTML:** parsed with BeautifulSoup; the title is extracted from standard page elements, and content is split into sections based on heading tags.
- **PDF:** parsed with Docling when available (with OCR support for scanned documents); otherwise PyMuPDF is used as a fallback.
- **DOCX:** parsed with `python-docx`, preserving heading structure and converting tables into text rows.
- **Images:** indexed via required sidecar `.txt/.md` descriptions (the image itself is not OCR'd).

To keep identifiers stable across machines and re-downloads, `doc_id` is derived preferentially from the canonical URL; if no URL is known, we fall back to content hashes. We also perform basic whitespace normalization and detect language (Slovenian/English) to drive prompting.

Chunking

Documents are chunked with a section-aware sliding window. Chunk size and overlap are measured in whitespace-split tokens (current configuration: 400-token chunks with 80-token overlap). Each chunk is contextualized by prefixing the document title and section name (when available), and receives a stable `chunk_id` of the form `<doc_id>_chunkXXXX`.

Embeddings and vector index

For semantic search we embed chunks with the multilingual E5 sentence-transformer (`intfloat/multilingual-e5-base`).

We apply E5-style prefixes (`passage:` for chunks and `query:` for questions) and L2-normalize embeddings. The index is built with FAISS `IndexFlatIP` (inner product); due to normalization this is equivalent to cosine similarity. For reproducibility, we persist the FAISS index together with chunk metadata (text, document title/section, URL, and scores).

Retrieval and reranking

Given a question, we compute its embedding and retrieve the top-*k* chunks from FAISS. The implementation also supports:

- **Hybrid retrieval:** BM25 lexical scores (Rank-BM25) are fused with dense scores on a candidate pool (union of lexical and dense top results), after min-max normalization of each score component.
- **Cross-encoder reranking:** an optional sentence-transformers CrossEncoder reranks the retrieved candidate set (default candidate pool size 20) and returns the final top-*k* list.

We additionally compute a simple `retrieval_weak` flag based on a score threshold; this is used both in the UI and in prompting as a warning signal when retrieval confidence is low.

Prompting and answer generation

For end-to-end question answering, retrieved chunks are formatted into a context block with numbered sources. A language-specific system prompt instructs the model to answer *only* from the provided context and to respond in the detected language (Slovenian or English). Generation uses Hugging Face Transformers causal language models with configurable decoding parameters (maximum new tokens, temperature, and nucleus sampling). The system can also run in retrieval-only mode, which is useful for debugging and evaluation without requiring GPU resources.

Evaluation setup

We evaluate on a JSONL file of manually written questions. Each question includes the expected language and expected keywords. Importantly, we maintain `relevant_chunk_ids` (and optionally `relevant_doc_ids`) for each positive question, i.e., the questions have already been indexed to the gold chunks containing the answer. This enables reporting standard retrieval metrics at a chosen *k* (chunk-level Recall@*k*, Precision@*k*, MRR@*k*, and nDCG@*k*). For end-to-end QA we additionally report keyword hit rate in generated answers, as well as retrieval and generation time per question. We also include negative (out-of-domain) questions to measure abstention behaviour via the `retrieval_weak` heuristic.

Equations

You can write equations inline, e.g. $\cos \pi = -1$, $E = m \cdot c^2$ and α , or you can include them as separate objects. The Bayes's rule is stated mathematically as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}, \quad (1)$$

where *A* and *B* are some events. You can also reference it – the equation 1 describes the Bayes's rule.

Lists

We can insert numbered and bullet lists:

1. First item in the list.
2. Second item in the list.

3. Third item in the list.

- First item in the list.
- Second item in the list.
- Third item in the list.

We can use the description environment to define or describe key terms and phrases.

Word What is a word?.

Concept What is a concept?

Idea What is an idea?

Random text

This text is inserted only to make this template look more like a proper report. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam blandit dictum facilisis. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Interdum et malesuada fames ac ante ipsum primis in faucibus. Etiam convallis tellus velit, quis ornare ipsum aliquam id. Maecenas tempus mauris sit amet libero elementum eleifend. Nulla nunc orci, consectetur non consequat ac, consequat non nisl. Aenean vitae dui nec ex fringilla malesuada. Proin elit libero, faucibus eget neque quis, condimentum laoreet urna. Etiam at nunc quis felis pulvinar dignissim. Phasellus turpis turpis, vestibulum eget imperdiet in, molestie eget neque. Curabitur quis ante sed nunc varius dictum non quis nisl. Donec nec lobortis velit. Ut cursus, libero efficitur dictum imperdiet, odio mi fermentum dui, id vulputate metus velit sit amet risus. Nulla vel volutpat elit. Mauris ex erat, pulvinar ac accumsan sit amet, ultrices sit amet turpis.

Phasellus in ligula nunc. Vivamus sem lorem, malesuada sed pretium quis, varius convallis lectus. Quisque in risus nec lectus lobortis gravida non a sem. Quisque et vestibulum sem, vel mollis dolor. Nullam ante ex, scelerisque ac efficitur vel, rhoncus quis lectus. Pellentesque scelerisque efficitur purus in faucibus. Maecenas vestibulum vulputate nisl sed vestibulum. Nullam varius turpis in hendrerit posuere.

Figures

You can insert figures that span over the whole page, or over just a single column. The first one, Figure 1, is an example of a figure that spans only across one of the two columns in the report.

On the other hand, Figure 2 is an example of a figure that spans across the whole page (across both columns) of the report.

Tables

Use the table environment to insert tables.

Code examples

You can also insert short code examples. You can specify them manually, or insert a whole file with code. Please avoid inserting long code snippets, advisors will have access to

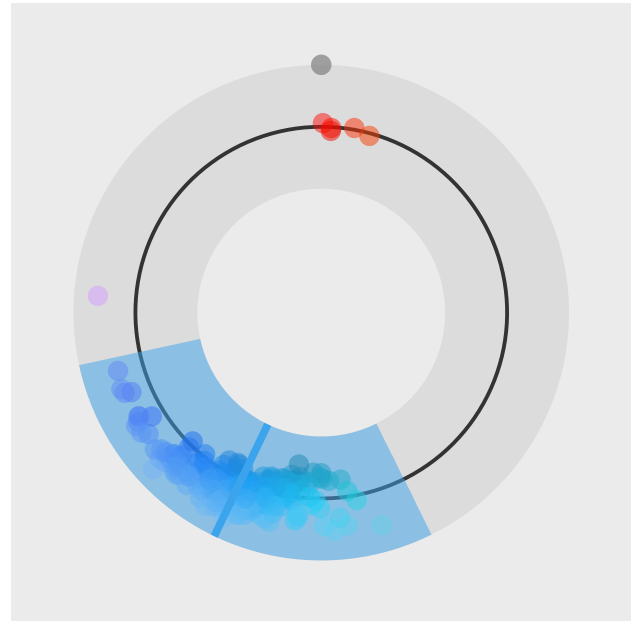


Figure 1. A random visualization. This is an example of a figure that spans only across one of the two columns.

Table 1. Table of grades.

Name		
First name	Last Name	Grade
John	Doe	7.5
Jane	Doe	10
Mike	Smith	8

your repositories and can take a look at your code there. If necessary, you can use this technique to insert code (or pseudo code) of short algorithms that are crucial for the understanding of the manuscript.

Listing 1. Insert code directly from a file.

```
import os
import time
import random

fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

Listing 2. Write the code you want to insert.

```
import (dplyr)
import (ggplot)

ggplot(diamonds,
       aes(x=carat, y=price, color=cut)) +
  geom_point() +
  geom_smooth()
```

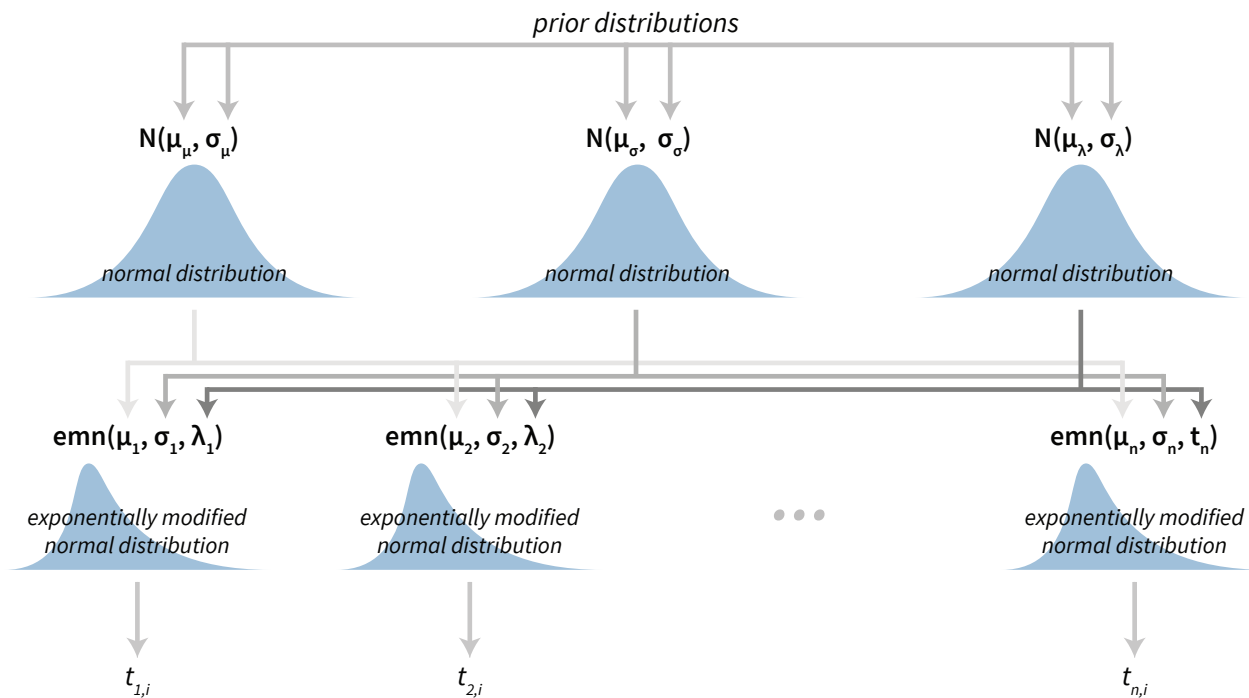


Figure 2. Visualization of a Bayesian hierarchical model. This is an example of a figure that spans the whole width of the report.

Results

Our approach correctly obtains, processes and builds a vector index from the collected data, and supports retrieval and (optionally) answer generation for both Slovenian and English questions. Since the evaluation questions are annotated with gold relevant chunks, we can report retrieval metrics directly and compare retrieval configurations (dense, hybrid, rerank).

Index statistics (to be filled)

Table 2. Run and index statistics (placeholders).

Statistic	Value
Run name	TBD
Seed URLs	TBD
Parsed documents (total)	TBD
Total chunks	TBD
Chunk size / overlap (tokens)	400 / 80
Embedding model	<i>intfloat/multilingual-e5-base</i>
FAISS index	<i>IndexFlatIP</i>

Retrieval metrics (annotated gold chunks)

End-to-end QA metrics (to be filled)

Qualitative examples

A small demonstrative evaluation with the *Qwen/Qwen2.5-1.5B-Instruct* model on an average laptop CPU shows that the model can answer questions in both Slovene and English,

and that the answers contain the expected keyword, indicating that the indexing and prompting pipeline is working.

Discussion

The discrepancy in the answer generation time is most likely either a result of fluctuating system load during the evaluation or a property of the model used for answering. At this stage, generation on CPU can be slow for longer outputs; for systematic comparisons we will run the evaluation on a controlled environment (HPC/GPU) and report averages (and optionally percentiles) over the full annotated question set. We also expect that hybrid retrieval and reranking will improve retrieval metrics, especially for short or ambiguous questions where lexical cues matter.

Acknowledgments

Here you can thank other persons (advisors, colleagues ...) that contributed to the successful completion of your project.

References

- [1] Eleni Adamopoulou and Lefteris Moussiades. An overview of chatbot technology. In *IFIP international conference on artificial intelligence applications and innovations*, pages 373–383. Springer, 2020.
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich

Table 3. Retrieval performance on the annotated evaluation set (chunk-level metrics; placeholders).

Retrieval setting	k	Recall@ k	Precision@ k	MRR@ k	nDCG@ k
Dense (E5 + FAISS)	4	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Hybrid (BM25 + dense fusion)	4	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + rerank (cross-encoder)	4	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Hybrid + rerank	4	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 4. End-to-end QA metrics (placeholders; fill after running evaluation).

Model	Retrieval setting	Keyword hit rate	Avg. retrieval time [s]	Avg. generation time [s]
Qwen/Qwen2.5-1.5B-Instruct	Dense	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
cjvt/GaMS3-12B-Instruct	Dense	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
meta-llama/Llama-3.1-8B-Instruct	Dense	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
mistralai/Mistral-7B-Instruct-v0.3	Dense	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[3] Bhavika R Ranoliya, Nidhi Raghuwanshi, and Sanjay Singh. Chatbot for university related faqs. In *2017 international conference on advances in computing, communications and informatics (ICACCI)*, pages 1525–1530. IEEE, 2017.

[4] Subash Neupane, Elias Hossain, Jason Keith, Himanshu Tripathi, Farbod Ghiasi, Noorbakhsh Amiri Golilarz, Amin Amirlatifi, Sudip Mittal, and Shahram Rahimi. From questions to insightful answers: Building an informed chatbot for university resources, 2024.

[5] Maksuda Khasanova Zafar kizi and Youngjung Suh.

Design and performance evaluation of llm-based rag pipelines for chatbot services in international student admissions. *Electronics*, 14(15), 2025.

[6] Akash Shanthakumar, Farhad Fassihi, Ahmad Lotfi, and Jordan Bird. *Retrieval Augmented Large Language Model Chatbots in Higher Education: A Study on University Open Days*, pages 32–44. 01 2025.

[7] Oded Ovadia, Menachem Brief, Moshik Mishaeli, and Oren Elisha. Fine-tuning or retrieval? comparing knowledge injection in llms, 2024.

[8] Zongxi Li, Zijian Wang, Weiming Wang, Kevin Hung, Haoran Xie, and Fu Lee Wang. Retrieval-augmented generation for educational application: A systematic survey. *Computers and Education: Artificial Intelligence*, 8:100417, 2025.

Question	Expected keywords	Answer contains all expected kw.	Generation time [s]
Kako poteka prijava na izpit?	STUDIS	yes	2153.03
How do I apply for an exam?	STUDIS	yes	384.21

Table 5. Qualitative example results (CPU run).